



ACTIVITY COMPLETE

# Great work, Abhiram Chittampally!

## 425

Total points scored

### Review your answers

16 of 20 correct



Q1

**Which SQL clause is used to filter which rows appear in the result set?**

Your answer: C. WHERE

**Correct answer: C. WHERE**

Q2

**You need customers whose phone value is missing (NULL). Which WHERE clause finds them?**

SQL

```
-- customers(id, name, phone)
SELECT * FROM customers WHERE ...;
```

Your answer: B. WHERE phone IS NULL

**Correct answer: B. WHERE phone IS NULL**

Q3

**Given the data below, what value does the query return?**

SQL

```
-- orders(id, customer_id):
-- (1, 10), (2, 20), (3, 20), (4, NULL), (5, 30)

SELECT COUNT(customer_id) FROM orders;
```

Your answer: C. 5

**Correct answer: B. 4**

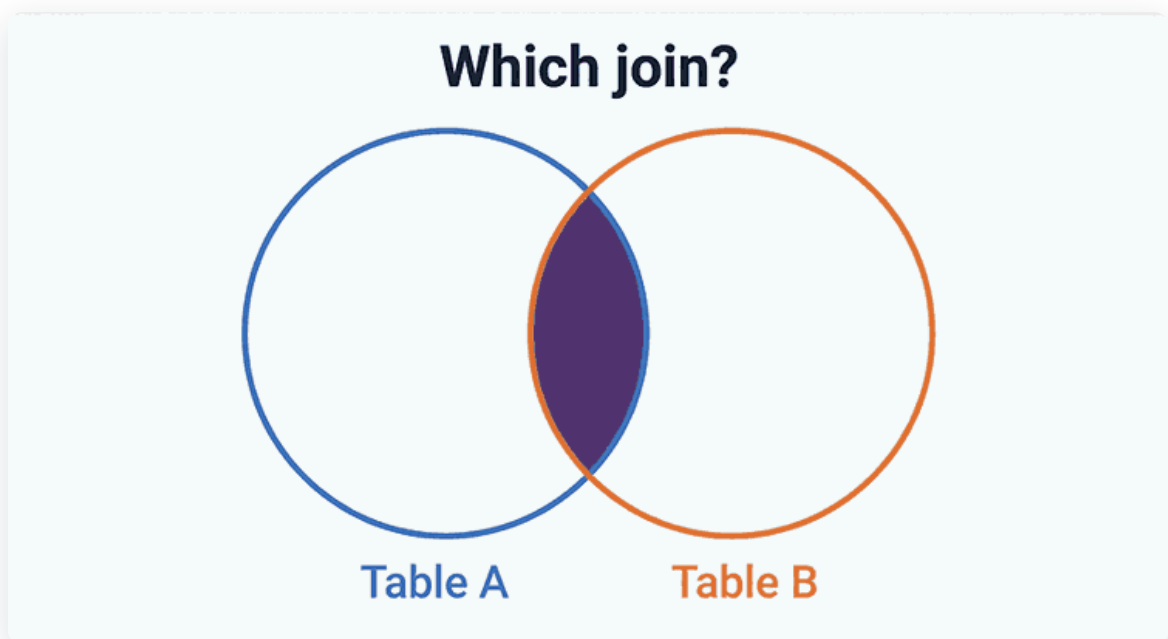
Q4

**Which statement correctly describes the difference between WHERE and HAVING?**

Your answer: A. WHERE filters rows before grouping; HAVING filters groups after aggregation.

**Correct answer: A. WHERE filters rows before grouping; HAVING filters groups after aggregation.**

Q5

**The shaded region in the diagram represents the result of which join type?****Image uploaded and ready****This is the image students will see.****Optimized image preview****Click Save to keep this image with the activity.**

Your answer: A. INNER JOIN

**Correct answer: A. INNER JOIN**

Q6

**Given the data, how many rows does the query return?****SQL**

```
-- customers(id, name):  
-- (1, 'Amy'), (2, 'Bob'), (3, 'Cid')  
-- orders(id, cust_id):  
-- (10, 1), (11, 1), (12, 2)  
  
SELECT c.name, o.id  
FROM customers c  
LEFT JOIN orders o ON o.cust_id = c.id;
```

Your answer: B. 3

**Correct answer: C. 4**

Q7

**You need customers who have NEVER placed an order. Which query implements the anti-join pattern correctly?**

SQL

```
-- customers(id, name)
-- orders(id, customer_id)
```

Your answer: B. SELECT c.\* FROM customers c LEFT JOIN orders o ON o.customer\_id = c.id WHERE o.id IS NULL;

✓

**Correct answer: B. SELECT c.\* FROM customers c LEFT JOIN orders o ON o.customer\_id = c.id WHERE o.id IS NULL;**

Q8

**Which query correctly returns products priced above the overall average price?**

SQL

```
-- products(id, name, price)
```

Your answer: B. SELECT \* FROM products WHERE price > (SELECT AVG(price) FROM products);

✓

**Correct answer: B. SELECT \* FROM products WHERE price > (SELECT AVG(price) FROM products);**

Q9

**What is the key difference between UNION and UNION ALL?**

Your answer: A. UNION removes duplicate rows; UNION ALL keeps all rows including duplicates.

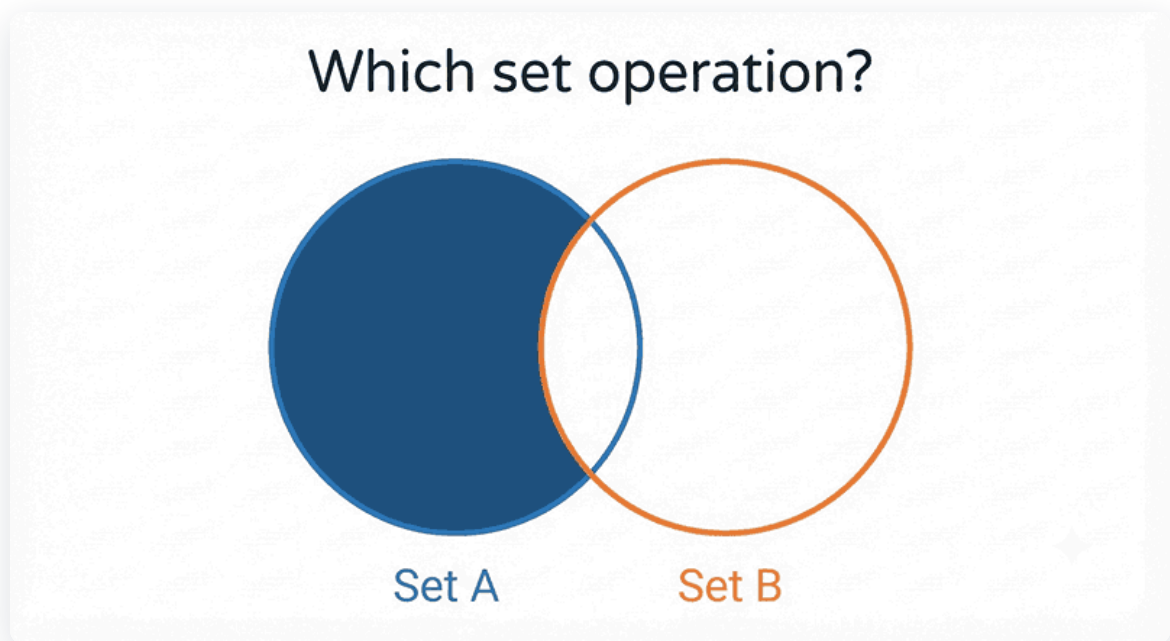
✓

**Correct answer: A. UNION removes duplicate rows; UNION ALL keeps all rows including duplicates.**

Q10

**The shaded region shows the result of which set operation?****Image uploaded and ready**

This is the image students will see.

**Optimized image preview**

Click Save to keep this image with the activity.

Your answer: C. A EXCEPT B

**Correct answer: C. A EXCEPT B**

Q11

**What is the primary purpose of a FOREIGN KEY constraint?**

Your answer: B. To ensure a column's value must exist in another table's referenced column.

**Correct answer: B. To ensure a column's value must exist in another table's referenced column.**

Q12

**Which column is the BEST candidate to place a B-Tree index on?**

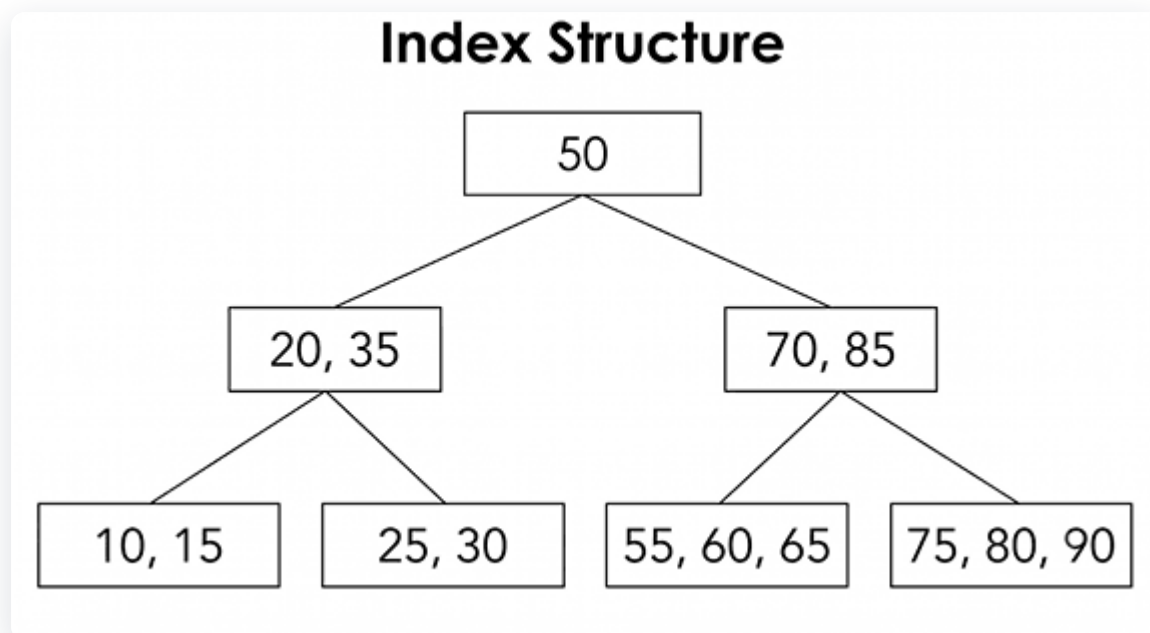
Your answer: A. A column frequently used in WHERE clauses for equality and range lookups.

**Correct answer: A. A column frequently used in WHERE clauses for equality and range lookups.**

Q13

**The diagram shows the internal structure of which index type?****Image uploaded and ready**

This is the image students will see.

**Optimized image preview**[Click Save to keep this image with the activity.](#)

Your answer: B. B-Tree index

**Correct answer: B. B-Tree index**

Q14

**The query planner chooses a SEQUENTIAL SCAN instead of an available index scan. Which situation makes this a reasonable choice?**

Your answer: A. The query returns a very small fraction of the table's rows.

**Correct answer: B. The query is expected to return a large fraction of the table's rows.**

Q15

**Which statement about SQL functions and stored procedures is TRUE?**

Your answer: A. Functions must return a value and can be used inside SELECT expressions; procedures are invoked with CALL and need not return a value.

**Correct answer: A. Functions must return a value and can be used inside SELECT expressions; procedures are invoked with CALL and need not return a value.**

Q16

**Which statement about MATERIALIZED VIEWS is TRUE?**

Your answer: A. A materialized view always shows the latest data with no refresh needed.

**Correct answer: B. A materialized view stores the query result physically and must be refreshed to reflect new base-table data.**

Q17

**Which query uses a Common Table Expression (CTE) with correct syntax?****SQL**

```
-- Goal: compute average salary per department,  
-- then list departments above the overall average.
```

Your answer: A. WITH dept\_avg AS (SELECT dept, AVG(salary) AS a FROM emp GROUP BY dept) SELECT \* FROM dept\_avg WHERE a > (SELECT AVG(salary) FROM emp);

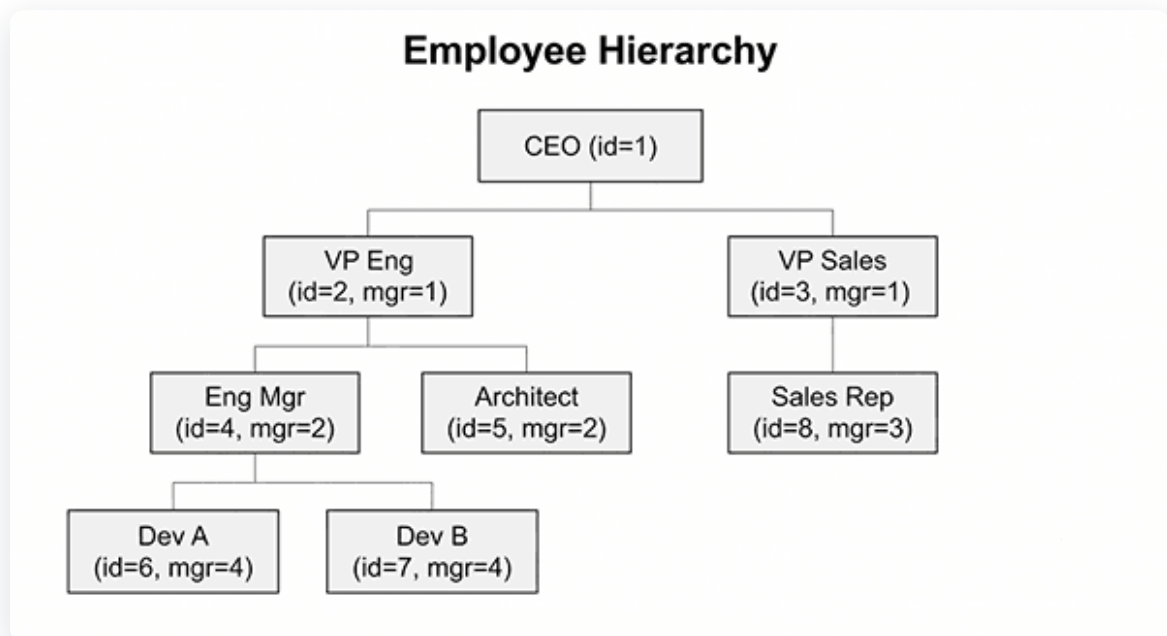
✓

Correct answer: A. WITH dept\_avg AS (SELECT dept, AVG(salary) AS a FROM emp GROUP BY dept) SELECT \* FROM dept\_avg WHERE a > (SELECT AVG(salary) FROM emp);

Q18

**To retrieve every employee under 'VP Eng (id=2)' — direct reports plus their reports at every depth — which SQL feature is the natural fit?**

✓

**Image uploaded and ready****This is the image students will see.**

✓

**Optimized image preview****Click Save to keep this image with the activity.**

Your answer: B. A recursive CTE

✓

Correct answer: B. A recursive CTE

Q19

**What values do the columns r and d return, in row order (Amy, Bob, Cid)?****SQL**

```
-- students(name, score):  
-- ('Amy', 90), ('Bob', 90), ('Cid', 85)  
  
SELECT  
  name,  
  RANK()    OVER (ORDER BY score DESC) AS r,  
  DENSE_RANK() OVER (ORDER BY score DESC) AS d  
FROM students;
```

Your answer: B. r: 1, 1, 3 d: 1, 1, 2

**Correct answer: B. r: 1, 1, 3 d: 1, 1, 2**

Q20

**What is the value of prev\_amt for the row where day = 3?****SQL**

```
-- sales(day, amt):  
-- (1, 100), (2, 150), (3, 120), (4, 200)  
  
SELECT  
  day,  
  amt,  
  LAG(amt) OVER (ORDER BY day) AS prev_amt  
FROM sales;
```

Your answer: C. 150

**Correct answer: C. 150**